
CHAPTER 8

Encipherment Using Modern Block Ciphers

(Solution to Practice Set)

Review Questions

1. Modern block ciphers encrypt and decrypt small blocks. DES uses a block size of 8 bytes (characters) and AES uses a block size of 16 bytes (characters). In real life, we need to encrypt or decrypt larger units of data. For example, a one-page document is normally more than 1000 characters. Mode of operations are designed to allow us to repeatedly use a modern block cipher for encryption and decryption.
2. We discussed electronic codebook (ECB), cipher block chaining (CBC), cipher feedback (CFB), output feedback (OFB), and counter (CTR) modes.
3. In the electronic codebook (ECB) mode, the plaintext is divided into N blocks. Each block is n bits. The same key is used to encrypt and decrypt each block.
 - ❑ **Advantages.** This mode has two obvious advantages. First, it is simple. Second, transmission error is not propagated from one block to the other.
 - ❑ **Disadvantages.** This mode has some security problems. First, patterns at the block level are preserved. Second, block independency creates opportunities for Eve to substitute some cipher blocks with some cipher blocks of her own.
4. In the cipher block chaining (CBC) mode, each plaintext block is exclusive-ored with the previous ciphertext block before being encrypted. A phony block called the initial vector (IV) is used to serve as C_0 . The same key is used to encrypt and decrypt each block.
 - ❑ **Advantages.** This mode has one obvious advantage. Each ciphertext block depends on previous ciphertext blocks. Patterns at the block level are not preserved. Eve cannot reorder the ciphertext blocks.
 - ❑ **Disadvantages.** This mode has some security problems. First, if two messages are equal, their encipherment is the same if they use the same IV. Second, Eve can add some ciphertext blocks at the end of the message. The mode also has some error-propagation problem: a single bit error in one ciphertext block may create errors in the corresponding plaintext block and the next plaintext block.

5. In the cipher feedback (CFB) mode, the size of the plaintext or ciphertext block is r , where $r \leq n$. The idea is to use DES or AES, not for encrypting the plaintext or decrypting the ciphertext, but to encrypt or decrypt the contents of a shift register, S , of size n . Data encryption is done by exclusive-oring an r -bit plaintext block with r bits of the shift register. Data decryption is done by exclusive-oring an r -bit ciphertext block with r bits of the shift register. For each block, the shift register S_i is made by shifting the shift register S_{i-1} (previous shift register) r bits to the left and filling the rightmost r bits with C_{i-1} .
 - ❑ **Advantages.** One advantage of CFB is that no padding is required because the size of the blocks, r , is normally chosen to fit the data unit to be encrypted. Another advantage is that the system does not have to wait until it has received a large block of data before starting the encryption.
 - ❑ **Disadvantages.** One disadvantage of CFB is that it is less efficient than CBC or ECB, because it needs to apply the encryption function of underlying block cipher for each small block of size r . Another disadvantage is that Eve can add some ciphertext block to the end of the stream.
6. The output feedback (OFB) mode is very similar to CFB mode, with one difference: each bit in the ciphertext is independent of the previous bit or bits. This avoids error propagation. If an error occurs in transmission, it does not affect the bits that follow. Like CFB, both the sender and the receiver use the encryption algorithm.
 - ❑ **Advantages.** One advantage of OFB is that no padding is required because the size of the blocks, r , is normally chosen to fit the data unit to be encrypted. Another advantage is that the system does not have to wait until it has received a large block of data before starting the encryption.
 - ❑ **Disadvantages.** One disadvantage of OFB is that it is less efficient than CBC or ECB, because it needs to apply the encryption function of underlying block cipher for each small block of size r . Another disadvantage is that Eve can add some ciphertext block to the end of the stream.
7. In the counter (CTR) mode, there is no feedback. The pseudorandomness in the key stream is achieved using a counter. An n -bit counter is initialized to a pre-determined value (IV) and incremented based on a predefined rule (mod 2^n). To provide a better randomness, the increment value can depend on the block number to be incremented. The plaintext and ciphertext block have the same block size as the underlying cipher (e.g., DEA or AES). Plaintext blocks of size n are encrypted to create ciphertext blocks of size n .
 - ❑ **Advantages.** One advantage of CTR is that it can be used to create random access file as long as the value of the counter is related to the record number in the file.
 - ❑ **Disadvantages.** One disadvantage of CTR is that the size of block is the same as the size of the underlying cipher (DES or AES). Encryption or decryption needs to be done n -bit at a time; the process needs to wait until n bit of data is accumulated.

8.

First Group: ECB and CBC**Second Group:** CFB, OFB, and CTR

9.

First Group: ECB and CBC**Second Group:** CFB, OFB, and CTR

10.

First Group: ECB and CBC**Second Group:** CFB, OFB, and CTR

11. RC4 uses a state of bytes for key generation. Each key in the key stream is a permutation of a key state. A5/1 uses the result of three LFSR's to create a key bit. We can say that key generation in RC4 is byte-oriented, but the key generation in A5/1 is bit-oriented. The other main difference is encryption and decryption. RC4 encrypts and decrypts a character at a time; A5/1 encrypts and decrypts a frame at a time.
12. The size of data in RC4 is normally 8 bits; data is normally encrypted or decrypted a byte at a time. The size of data in A5/1 is 228 bits; data is encrypted or decrypted a frame at a time.
13. ECB can be used for parallel processing because each block is encrypted or decrypted independently.
14. ECB can be used for encipherment of blocks in a random-access file because the encryption and decryption of each block is independent from the rest of the blocks.

Exercises

15. In CFB mode, the key generator for block i uses C_{i-1} , the ciphertext created in block $(i - 1)$. According to our definition in Chapter 5, this is a *nonsynchronous stream cipher*. In OFB mode, the key generator for block i uses part of the key from the previous block, but it is independent from the plaintext and ciphertext in the previous block. According to our definition in Chapter 5, this is a *synchronous stream cipher*.
16. In CFB, ciphertext is transmitted in blocks of r bits. A single bit error in a ciphertext block affects the corresponding bit plaintext block. However, the corruption does not stop here. The corrupted ciphertext block in the receiver site enters the shift register and corrupts the subsequent plaintext blocks until it falls off the shift register. In general, a single bit error in ciphertext, may corrupt $(n/r + 1)$ blocks of the plaintext. To see the relations, assume the underlying cipher is DES ($n = 64$) and encryption/decryption is done a byte at a time ($r = 8$). Imagine the second bit in ciphertext block 5 is corrupted during transmission. When this block is decrypted, only the second bit in plaintext block 5 is in error. However, this ciphertext block enters the shift register and remains in there until the next 8 blocks

arrives. This results in the possible corruption of all bits in the next 8 plaintext blocks. In other words, 9 blocks may have been corrupted.

17. In ECB, each block is decrypted independently. However, since the decryption is done a block at a time (DES or AES), the corrupted bit 17 in ciphertext block 8 may affect all n bits in plaintext block 8.
18. In CBC, if bits 17 and 18 in ciphertext block 9 are corrupted, all n bits in plaintext block 9 may be corrupted (decryption is one block at a time). However, bits 17 and 18 in ciphertext block 9 are also exclusive-ored with bits 17 and 18 of the next block (block 10). Therefore, $n + 2$ bits may be corrupted; n bits in block 9 and 2 bits in block 10. None of the bits of other blocks (block 11 ...) are affected. The system recovers itself after block 11.
19. As discussed in the solution to Exercise 16, a corrupted ciphertext block in CFB affects the corresponding plaintext block and the following n/r plaintext blocks. If we assume $n = 64$, the following plaintext blocks may be corrupted: block 11 (bits 3 to 6) and block 12 to 19 (all bits).
20. In CRT mode, there is no feedback. If ciphertext blocks 3 and 4 are corrupted, only plaintext blocks 3 and 4 will be corrupted.
21. In OFB mode, the feedback is only in the key-generation system. If ciphertext block 11 is corrupted, only plaintext block 11 may be corrupted.
22. In CFB mode, it is obvious that if k_i used at the Bob's site is the same as k_i used at the Alice's site, then $(P_i)'$ created by Bob is the same as P_i sent by Alice (assuming no corruption in transmission):

$$(P_i)' = (C_i)' \otimes k_i = P_i \otimes k_i \otimes k_i = P_i \otimes 0 = P_i$$

However, we also need to prove that k_i used by Alice and Bob are also the same. If there is no corruption in transmission and the IV's used by Alice and Bob are the same, then $(S_i)' = S_i$. Now, we can prove

$$(k_i)' = \text{ExtractLeft}_r [E_K(S_i)'] = \text{ExtractLeft}_r [E_K(S_i)] = k_i$$

23. In OFB mode, it is obvious that if k_i used at the Bob's site is the same as k_i used at the Alice's site, then $(P_i)'$ created by Bob is the same as P_i sent by Alice (assuming no corruption in transmission):

$$(P_i)' = (C_i)' \otimes k_i = P_i \otimes k_i \otimes k_i = P_i \otimes 0 = P_i$$

The k_i used by Bob is definitely the same as k_i used by Alice if both Alice and Bob use the same IV's.

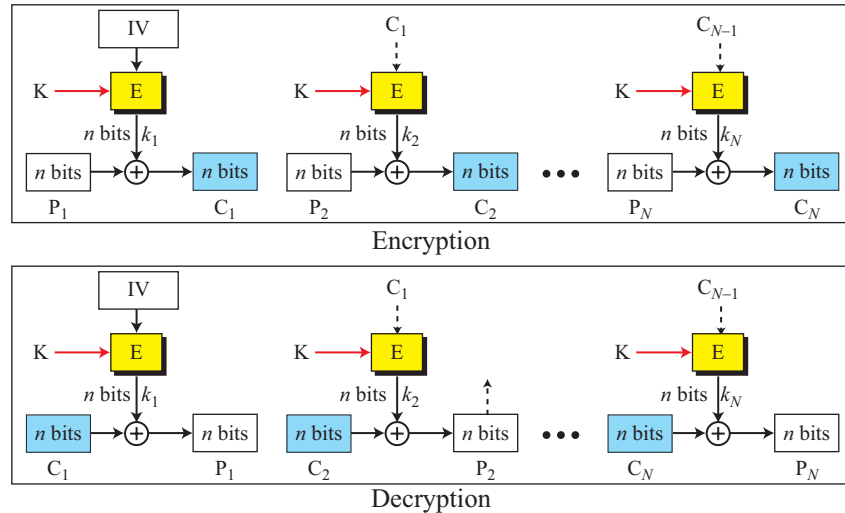
24. In CRT mode, it is obvious that if k_i used at the Bob's site is the same as k_i used at the Alice's site, then $(P_i)'$ created by Bob is the same as P_i sent by Alice (assuming no corruption in transmission):

$$(P_i)' = (C_i)' \otimes k_i = P_i \otimes k_i \otimes k_i = P_i \otimes 0 = P_i$$

The k_i used by Bob is definitely the same as k_i used by Alice if both Alice and Bob use the same IV's for the counter.

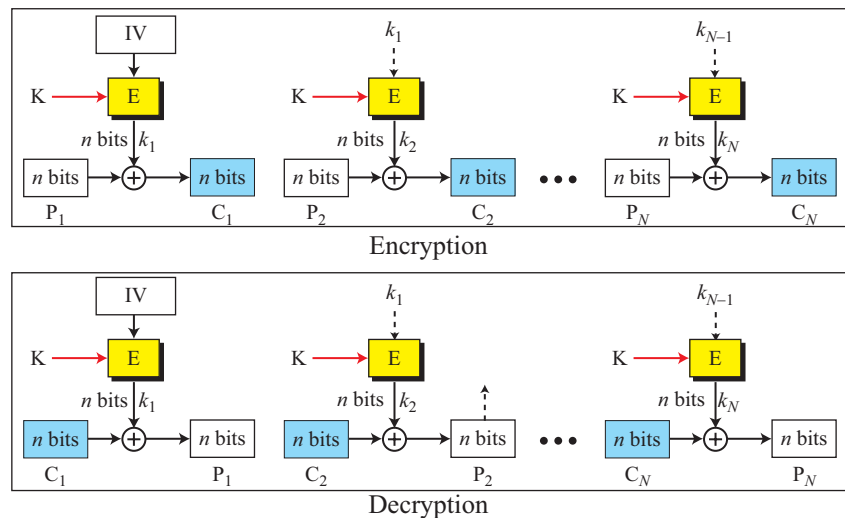
25. Figure S8.25 shows both the encryption and decryption. It is possible to cancel the underlying encryption for the first block and exclusive-or the IV directly with P_1 .

Figure S8.25 Solution to Exercise 25



26. Figure S8.26 shows both the encryption and decryption. It is possible to cancel the underlying encryption for the first block and exclusive-or the IV directly with P_1 .

Figure S8.26 Solution to Exercise 26

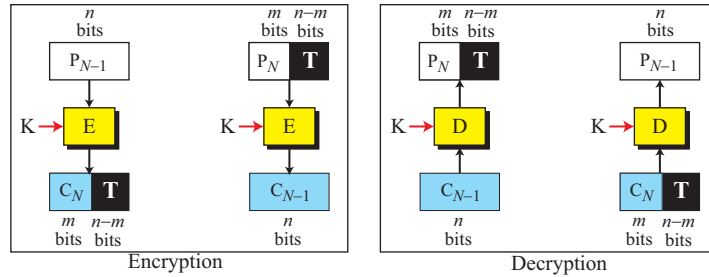


27. The following shows the process:

$$\begin{array}{ll} X = D_K(C_{N-1}) & \rightarrow P_N = \text{head}_m(X) \\ Y = C_N \mid \text{tail}_{n-m}(X) & \rightarrow P_{N-1} = D_K(Y) \end{array}$$

28. Figure S8.28 shows the diagram for encryption and decryption.

Figure S8.28 Solution to Exercise 28

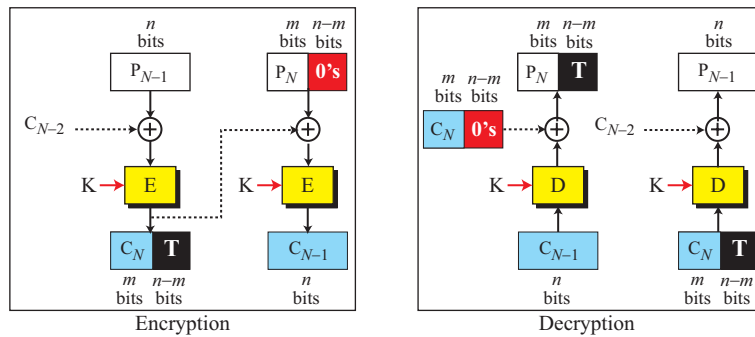


29. The following shows the process:

$$\begin{array}{ll} U = D_K(C_{N-1}) & X = U \oplus [C_N \mid \text{Pad}_{n-m}(0)] \rightarrow P_N = \text{head}_m(X) \\ Y = D_K[C_N \mid \text{tail}_{n-m}(X)] & \rightarrow P_{N-1} = C_{N-2} \oplus Y \end{array}$$

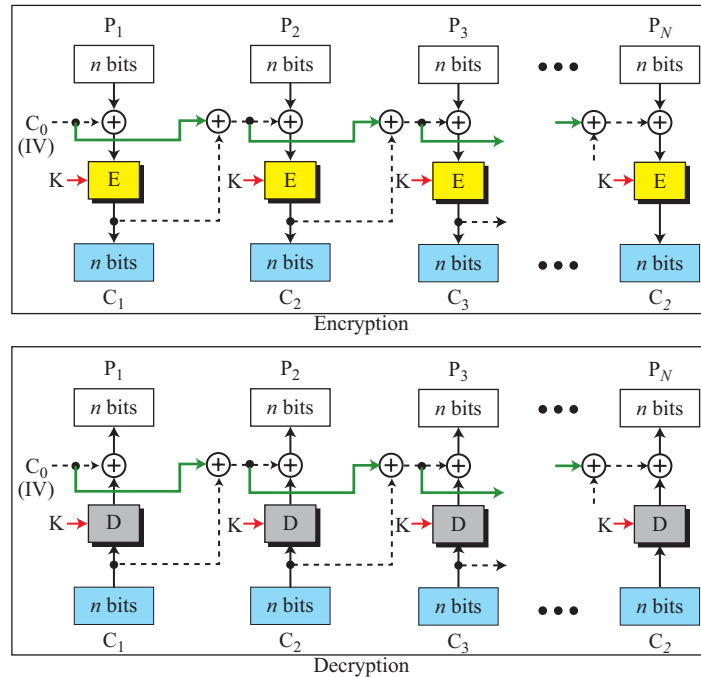
30. Figure S8.30 shows the diagram for encryption and decryption.

Figure S8.30 Solution to Exercise 30

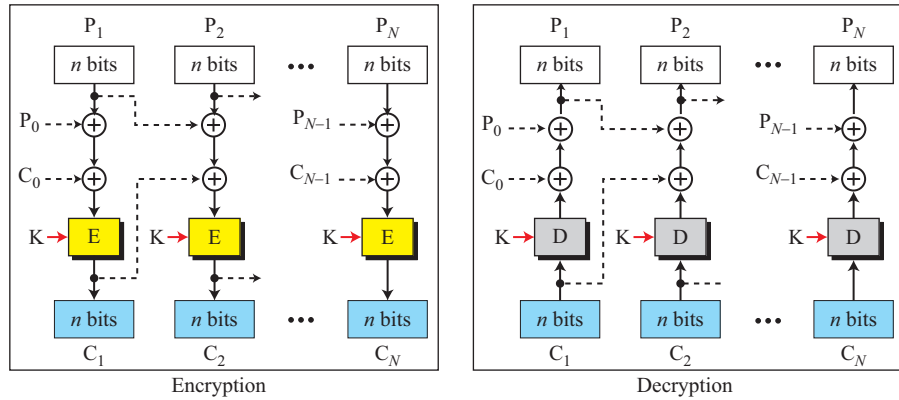
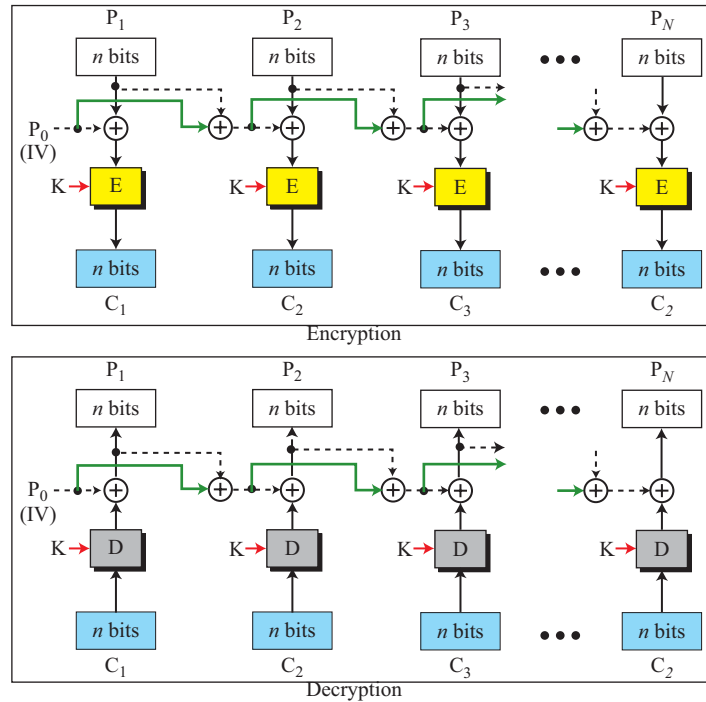


31. CFB, OFB, and CRT are stream ciphers, in which the size of the block is usually fixed (one character). There is no need for padding. This means there is no need for cipher stealing technique.
32. In this case, the error propagation is the same as the case where the CTS technique is not used except for the last two blocks. If C_{N-1} is corrupted, it corrupts both P_{N-1} and P_N . If C_N is corrupted, it corrupts only P_{N-1} .
33. In this case, the error propagation is the same as the case where the CTS technique is not used except for the last two blocks. If C_{N-1} is corrupted, it corrupts both P_{N-1} and P_N . If C_N is corrupted, it corrupts only P_{N-1} .
34. Figure S8.34 shows the BC mode.

Figure S8.34 Solution to Exercise 34



35. Figure S8.35 shows the PCBC mode. In PCBC an error in a ciphertext block corrupts all plaintext block that follows. This mode was used in Kerberos (See Chapter 15) to create both encryption and integrity of blocks. If a block was corrupted, all previous blocks were discarded.
36. Figure S8.36 shows the CBCC mode. In CBCC an error in a plaintext block corrupts all ciphertext block that follows.

Figure S8.35 Solution to Exercise 35**Figure S8.36** Solution to Exercise 36

37. We have written a program based on Algorithm 8.6 of the textbook. The result is

41 63 2 212 127 55 201 182 51 242 175 82 133 254 180 107 230 32 241 57

38. This situation is impossible because
- If the state must remain the same after the second step, it is needed that $S[i]$ has the same value for each $i = 0$ to 255 (for example all 0's, all 1's, all 2's, ...) because only in these cases the permutation in the following steps cannot change the values.
 - But the above condition can never happen because, before the second step, we have $S[0] = 0, S[1] = 1, \dots, S[255] = 255$ and the swapping can never make them the same.
39. This problem can be solved using the classical third birthday problem that we review in Appendix E. We have a sample set of k values, in which each sample can take one of the N values. What is the minimum size of k such that it is probable (with probability $P \geq 1/2$) such that at least two samples are the same. The answer is $k = 1.18 N^{1/2}$. In this case $N = 2^{128}$ possible keys, which means $k = 1.18 \times 2^{64}$.
40. According to the literature, all of the three characteristic polynomials are irreducible and the number of taps are even, so the maximum period can be found using the formula $2^m - 1$.
- For LFSR1, we have $2^{19} - 1$.
 - For LFSR2, we have $2^{22} - 1$.
 - For LFSR3, we have $2^{23} - 1$.
- 41.
- Majority (1, 0, 0) = 0. Therefore, LFSR₂ and LFSR₃ whose clocking bits matches with the value of the Majority function are clocked.
 - Majority (0, 1, 1) = 1. Therefore, LFSR₂ and LFSR₃ whose clocking bits matches with the value of the Majority function are clocked.
 - Majority (0, 0, 0) = 0. Therefore all three LFSR's are clocked.
 - Majority (1, 1, 1) = 1. Therefore all three LFSR's are clocked.
42. If the three bits in the Majority functions are called a , b , and c respectively, then

$$\text{Majority}(a, b, c) = (a \text{ AND } b) \oplus (b \text{ AND } c) \oplus (c \text{ AND } a)$$

43.

```
ECB_Decryption (K, C[1 ... N])
{
    for (i = 1 to N)
    {
        P[i] ← DK (C[i])
    }
    return (P[1 ... N])
}
```

44.

```

CBC_Decryption (IV, K, C[1 ... N])
{
    C[0]  $\leftarrow$  IV
    for ( $i = 1$  to  $N$ )
    {
        temp  $\leftarrow$   $D_K$  (C[i])
        temp  $\leftarrow$  temp  $\oplus$  C[i - 1]
    }
    return (P[1 ... N])
}

```

45. We use a variable Pre_C to hold the previous cipher block. In this way, we do not have to keep an array of ciphertext blocks.

```

CFB_Decryption (IV, K, r)
{
     $i \leftarrow 1$ 
    while (more blocks to decrypt)
    {
        input (C)
        if ( $i = 1$ )
            S  $\leftarrow$  IV
        else
        {
            Temp  $\leftarrow$  shiftLeft(r, S)
            S  $\leftarrow$  concatenate(Temp, Pre_C)
        }
        T  $\leftarrow$   $E_K$ (S)
         $k \leftarrow$  selectLeft(r, T)
        P  $\leftarrow$  C  $\oplus$  k
        output (P)
        Pre_C  $\leftarrow$  C
         $i \leftarrow i + 1$ 
    }
}

```

46. We use a variable `Pre_k` to hold the previous key. In this way, we do not have to keep an array of k 's.

```

OFB_Decryption (IV, K, r)
{
     $i \leftarrow 1$ 
    while (more blocks to decrypt)
    {
        input (C)
        if ( $i = 1$ )
             $S \leftarrow \text{IV}$ 
        else
        {
             $\text{Temp} \leftarrow \text{shiftLeft}(r, S)$ 
             $S \leftarrow \text{concatenate}(\text{Temp}, \text{Pre}_k)$ 
        }
         $T \leftarrow E_K(S)$ 
         $k \leftarrow \text{selectLeft}(r, T)$ 
         $P \leftarrow C \oplus k$ 
        output (P)
         $\text{Pre}_k \leftarrow k$ 
         $i \leftarrow i + 1$ 
    }
}

```

47. We have written the decryption algorithm assuming all N blocks are ready for decryption to match the encryption algorithm in the text. However, as described in the text, the algorithm can be written if ciphertext blocks are ready only one at a time.

```

CTR_Decryption (IV, K, C[1 ... N])
{
    Counter  $\leftarrow \text{IV}$ 
    for ( $i = 1$  to  $N$ )
    {
        Counter  $\leftarrow (\text{Counter} + i - 1) \bmod 2^N$ 
         $k_i \leftarrow E_K(\text{Counter})$ 
         $P[i] \leftarrow C[i] \oplus k[i]$ 
    }
    return (P[1 ... N])
}

```

48.

```
shiftLeft ( $r, S[1 \dots n]$ )  
{  
    for ( $i = n$  downto  $r$ )  
         $S[i] \leftarrow S[i - r]$   
    for ( $i = 0$  to  $r$ )  
         $S[i] \leftarrow 0$   
    return  $S$   
}
```

49.

```
concatenate ( $X[1 \dots n], Y[1 \dots r]$ )  
{  
    for ( $i = 1$  to  $r$ )  
         $X[n - r + 1] \leftarrow Y[i]$   
    return  $X$   
}
```

50.

```
selectLeft ( $r, X[1 \dots n]$ )  
{  
    for ( $i = 1$  to  $r$ )  
         $Y[i] \leftarrow X[i]$   
    return  $Y$   
}
```